

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Nayla Rizkia

NIM. 2510817120007

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan: Nayla Rizkia
NIM : 2510817120007

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
SOAL 1: Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code.....	3
B. Output Program	4
C. Pembahasan	4
SOAL 2: Konversi Adjacency Matrix ke Adjacency List.....	7
A. Source Code.....	9
B. Output Program	10
C. Pembahasan	10
SOAL 3: BFS: Jarak Terpendek Keluar Labirin	13
A. Source Code.....	15
B. Output Program	17
C. Pembahasan	17
SOAL 4: Menghitung Semua Jalur Keluar Labirin.....	19
A. Source Code.....	21
B. Output Program	22
C. Pembahasan	23
SOAL 5: Tree Traversal	25
A. Source Code.....	26
B. Output Program	29
C. Pembahasan	29
TAUTAN GIT	31

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	4
Gambar 2. Screenshot Hasil Jawaban Soal 2	10
Gambar 3. Screenshot Hasil Jawaban Soal 3	17
Gambar 4. Screenshot Hasil Jawaban Soal 4	22
Gambar 5. Screenshot Hasil Jawaban Soal 5	29

DAFTAR TABEL

Tabel 1. Source Code konversi1.cpp	3
Tabel 2. Source Code konversi2.cpp	9
Tabel 3. Source Code bfs.cpp	15
Tabel 4. Source Code dfs.cpp	21
Tabel 5. Source Code treeTraversal.cpp	26

SOAL 1:

Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut:

Adjacency Matrix:

$V_1 V_2 \dots V_N$

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki **self-loop**.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency Matrix:
A B C D E	A B C D E
6	A 0 4 2 0 0
A B 4	B 0 0 0 7 0
A C 2	C 0 1 0 0 6
B D 7	D 0 0 3 0 0
C B 1	E 0 0 0 0 0
C E 6	
D C 3	

Penjelasan Contoh:

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap selama tidak ada edge dari B menuju A. Edge D C 3

menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0.

A. Source Code

Tabel 1. Source Code konversi1.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	vector<char> vertices(N);
10	
11	for (int i = 0; i < N; i++) {
12	cin >> vertices[i];
13	}
14	
15	int M;
16	cin >> M;
17	
18	vector<vector<int>> matrix(N, vector<int>(N, 0));
19	
20	for (int i = 0; i < M; i++) {
21	char startVertex, endVertex;
22	int weight;
23	cin >> startVertex >> endVertex >> weight;
24	
25	int startIndex = 0, endIndex = 0;
26	
27	for (int j = 0; j < N; j++) {
28	if (vertices[j] == startVertex) {
29	startIndex = j;
30	}
31	if (vertices[j] == endVertex) {
32	endIndex = j;
33	}
34	}
35	matrix[startIndex][endIndex] = weight;
36	}

```

37
38     cout << "Adjacency Matrix:" << endl;
39
40     cout << " ";
41     for (int i = 0; i < N; i++) {
42         cout << vertices[i] << " ";
43     }
44     cout << endl;
45
46     for (int i = 0; i < N; i++) {
47         cout << vertices[i] << " ";
48         for (int j = 0; j < N; j++) {
49             cout << matrix[i][j] << " ";
50         }
51         cout << endl;
52     }
53
54     return 0;
55 }

```

B. Output Program



```

PS C:\Users\Lenovo\Documents\Github\Module-6-graph-and-tree> cd 'c:\Users\Lenovo\Documents\Github\Module-6-graph-and-tree\output'
PS C:\Users\Lenovo\Documents\Github\Module-6-graph-and-tree\output> & .\konversi1.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\Lenovo\Documents\Github\Module-6-graph-and-tree\output>

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Dalam membangun model directed weighted graph, representasi awal graf yang diberikan dalam bentuk himpunan edge dengan label berupa karakter harus diubah menjadi bentuk matriks grid secara matematis. Masalah utama dalam proses konversi ini adalah matriks hanya dapat dijalankan dan diakses melalui indeks numerik, sedangkan identitas setiap node didefinisikan dengan menggunakan karakter alfabet yang bersifat

unik. Oleh karena itu, dibutuhkan suatu metode pemetaan untuk mengubah karakter vertex menjadi indeks baris dan kolom yang valid pada matriks.

Untuk mengatasi perbedaan tersebut, algoritma dibuat dengan cara menyimpan setiap karakter vertex secara berurutan dalam struktur data berupa vector satu dimensi. Setelah indeks referensi ini dibuat, Adjacency Matrix dibentuk dengan menggunakan vector dua dimensi (2D) yang memiliki ukuran $N \times N$. Matriks ini diisi secara penuh dengan nilai nol, yang pada dasarnya menunjukkan tidak adanya jalur antar vertex. Saat sistem memproses daftar edge, program harus mengalokasikan lokasi matriks yang benar; hal ini dilakukan dengan melakukan pencarian sekuensial di dalam vector referensi vertex untuk menemukan karakter asal dan karakter tujuan yang sesuai. Setelah indeks integer ditemukan, bobot dari edge yang terkait langsung dimasukkan ke dalam Adjacency Matrix pada posisi baris dan kolom yang sesuai.

Sebagai contoh dalam proses ini, sebuah graf yang memiliki tiga vertex, yaitu A, B, dan C. Vector referensi akan menempatkan A pada indeks 0, B pada indeks 1, dan C pada indeks 2. Saat memproses instruksi masukan seperti "A B 4", proses mencari akan mengidentifikasi A pada baris 0 dan B pada kolom 1. Program selanjutnya akan memasukkan nilai 4 secara tepat ke dalam posisi `matrix[0][1]`. Karena graf yang digunakan bersifat directed, maka alur sebaliknya, yakni sel yang terletak pada baris B dan kolom A tidak akan menerima perubahan nilai apa pun dan tetap akan menunjukkan bahwa tidak ada jalur yang tersedia, sehingga nilainya tetap 0.

Analisis Kompleksitas

1. Kompleksitas Waktu: $O(N^2 + M \times N)$

Pembangunan algoritma ini memerlukan beberapa tahapan iterasi. Proses pembuatan dan pengaturan awal Adjacency Matrix membutuhkan iterasi bertingkat dengan kompleksitas sebesar $O(N^2)$. Selanjutnya, dalam mengatur bobot untuk setiap edge sebanyak M , dilakukan pengulangan pencarian ke dalam daftar vertex dengan biaya waktu sebesar $O(M \times N)$. Karena batasan soal yang diberikan cukup terbatas, yaitu $N \leq 10$ dan jumlah M maksimal sekitar $10 \times 9 = 90$ operasi, maka perhitungan beban kerja maksimal hanya mencapai puluhan hingga ratusan instruksi saja. Ini sangat efisien dan eksekusi akan berlangsung segera, jauh di bawah Time Limit 1.0 s.

2. Kompleksitas Ruang: $O(N^2)$

Kebutuhan penyimpanan data utama secara pasti didominasi oleh pembuatan Adjacency Matrix berbentuk vektor dua dimensi yang mengalokasikan ruang memori berupa bilangan bulat sebanyak $N \times N$ blok. Juga diperlukan alokasi array satu dimensi dengan kompleksitas $O(N)$ untuk menyimpan label karakter dari setiap vertex. Dengan batasan maksimum $N=10$, ruang terbesar yang dialokasikan dalam memori program tidak akan melebihi jumlah 100 nilai integer. Skala operasi struktur data ini secara keseluruhan bersifat statis dan minimal, sehingga berjalan dengan stabil dan aman untuk dijalankan dalam lingkungan batasan Memory Limit 64 MB.

SOAL 2: Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut:

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .

- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut:

Adjacency List: $V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

$V_N \rightarrow \dots$

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol $\rightarrow$.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0	Adjacency List: A $\rightarrow$ (B,4), (C,2) B $\rightarrow$ (D,7) C $\rightarrow$ (B,1), (E,6)

0 1 0 0 6	D -> (C,3)
0 0 3 0 0	E -> -
0 0 0 0 0	

Penjelasan Contoh:

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2). Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2. Source Code konversi2.cpp

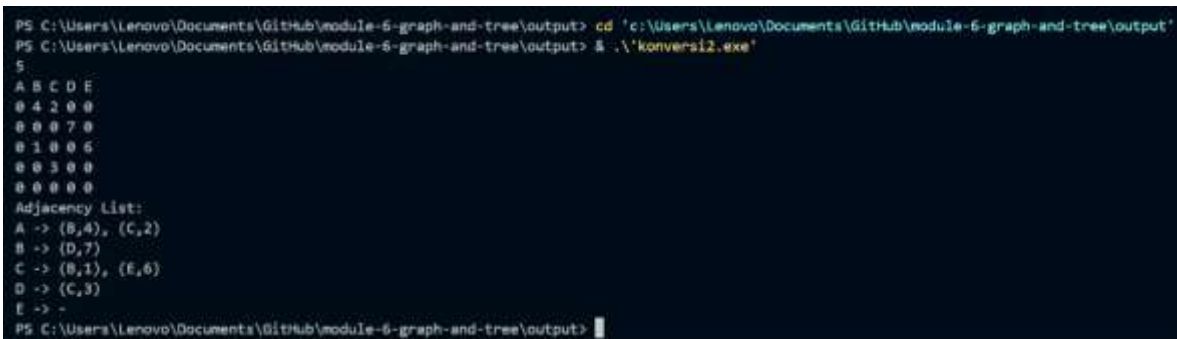
1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	vector<char> vertices(N);
10	for (int i = 0; i < N; i++) {
11	cin >> vertices[i];
12	}
13	
14	vector<vector<int>> matrix(N, vector<int>(N));
15	
16	for (int i = 0; i < N; i++) {
17	for (int j = 0; j < N; j++) {
18	cin >> matrix[i][j];
19	}
20	}
21	
22	cout << "Adjacency List:" << endl;
23	
24	for (int i = 0; i < N; i++) {
25	cout << vertices[i] << " -> ";
26	
27	bool hasEdge = false;
28	bool isFirst = true;

```

29
30         for (int j = 0; j < N; j++) {
31             if (matrix[i][j] > 0) {
32                 hasEdge = true;
33
34                 if (!isFirst) {
35                     cout << ", ";
36                 }
37                 cout << "(" << vertices[j] << ", " <<
matrix[i][j] << ")";
38                 isFirst = false;
39             }
40         }
41
42         if (!hasEdge) {
43             cout << "-";
44         }
45         cout << endl;
46     }
47
48     return 0;
49 }

```

B. Output Program



```

PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> cd 'c:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output'
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> & .\konversi2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (D,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output>

```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Latar belakang permasalahan ini didasari oleh kebutuhan untuk mengubah struktur data directed weighted graph dari bentuk Adjacency Matrix menjadi daftar ketetanggaan. Matriks statis menyimpan ketiadaan hubungan dengan nilai 0, yang dapat menghabiskan banyak memori jika graf tersebut memiliki sedikit edge (sparse graph). Oleh karena itu,

tujuan utamanya adalah mengekstraksi dan merangkum semua koneksi aktif, yaitu jalur yang memiliki bobot lebih dari nol, sehingga setiap vertex hanya berisi daftar simpul tujuan yang relevan dan dapat dicapai secara langsung.

Proses ekstraksi dilakukan dengan langkah pertama memetakan label karakter pada setiap vertex ke dalam struktur vector satu dimensi agar indeks numerik dapat diformat kembali menjadi huruf alfabet. Matriks dibaca secara lengkap dan disimpan ke dalam bentuk Vector 2D. Algoritma kemudian memproses data secara bertahap, lapis demi lapis, berdasarkan setiap baris dalam matriks, di mana setiap baris mewakili vertex asal. Dalam setiap baris, program memeriksa setiap kolom secara berurutan; jika ditemukan nilai yang lebih besar dari 0, maka hal tersebut dianggap sebagai adanya sebuah edge beserta bobotnya. Data ini kemudian diatur menggunakan variabel bantuan berupa tipe boolean untuk mengontrol penambahan tanda koma yang terstruktur di antara setiap elemen, serta mengidentifikasi kondisi khusus di mana jika suatu vertex tidak memiliki edge yang keluar, maka output akan diganti dengan simbol hubung (-).

Sebagai contoh, jika dalam pemindaian baris indeks ke-0 (yang mewakili vertex A) terdapat nilai positif 4 pada kolom indeks ke-1 (yang mewakili B) dan nilai 2 pada kolom indeks ke-2 (yang mewakili C), maka algoritma akan menghasilkan string dengan urutan A -> (B,4), (C,2). Sebaliknya, jika pemindaian dilakukan pada baris vertex E dan seluruh isi kolom pada baris tersebut terbukti bernilai 0, maka algoritma menyimpulkan bahwa tidak ada edge yang keluar, sehingga proses pencetakan langsung dialihkan menjadi E -> -.

Analisis Kompleksitas

1. Kompleksitas Waktu: $O(N^2)$

Operasi utama dalam program ini adalah membaca struktur Adjacency Matrix dan melakukan pemindaian berulang dengan menggunakan iterasi bersarang. Loop luar berfungsi untuk memproses setiap baris sebanyak N kali, sedangkan di dalam loop luar tersebut, loop dalam berjalan untuk memproses setiap kolom sebanyak N kali. Dengan batasan soal yang menentukan nilai $N \leq 10$, jumlah maksimal siklus eksekusi hanya akan mencapai angka $10 \times 10 = 100$ iterasi yang tetap konstan. Angka ini sangat kecil sehingga dapat diproses dengan cepat oleh CPU modern,

sehingga eksekusi program dapat selesai hampir seketika dan sangat cocok untuk memenuhi batas waktu 1,0 detik yang ditetapkan.

2. Kompleksitas Ruang: $O(N^2)$

Faktor utama yang menyebabkan penggunaan memori besar adalah penyimpanan Adjacency Matrix dalam bentuk vector dua dimensi dengan ukuran $N \times N$ yang menyimpan integer. Tersedia alokasi sekunder berupa satu vector satu dimensi dengan ukuran N untuk mencatat label karakter, tetapi peran vector tersebut tidak mendominasi secara asimtotik. Berdasarkan batasan $N \leq 10$, jumlah blok memori yang dibutuhkan program ini untuk elemen matriks tidak akan melebihi 100 blok. Penggunaan sumber daya ini termasuk sangat statis, berada jauh di bawah batas aman, dan tidak akan mengganggu kapasitas memori dengan batas 64 MB.

SOAL 3:

BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut:

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut: R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat: X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1.

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 0 1 1 0 0 0 0 0 0 0 0 1 1 1 0 1 1 1 1 0 0 1 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 0 1	16

1 1 0 0 0 0 1 0 0 0	
0 0	
7 9	

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3. Source Code bfs.cpp

1	#include <iostream>
2	#include <vector>
3	#include <queue>
4	using namespace std;
5	
6	struct Position {
7	int row;
8	int col;
9	int distance;
10	};
11	
12	int main() {
13	int R, C;
14	cin >> R >> C;
15	
16	vector<vector<int>> grid(R, vector<int>(C));
17	for (int i = 0; i < R; i++) {
18	for (int j = 0; j < C; j++) {
19	cin >> grid[i][j];
20	}
21	}
22	
23	int startRow, startCol, endRow, endCol;
24	cin >> startRow >> startCol;
25	cin >> endRow >> endCol;
26	

```

27     queue<Position> q;
28     q.push({startRow, startCol, 0});
29
30     vector<vector<bool>> visited(R, vector<bool>(C,
31 false));
32     visited[startRow][startCol] = true;
33
34     int dRow[] = {-1, 1, 0, 0};
35     int dCol[] = {0, 0, -1, 1};
36
37     int shortestPath = -1;
38
39     while (!q.empty()) {
40         Position current = q.front();
41         q.pop();
42
43         if (current.row == endRow && current.col ==
44 endCol) {
45             shortestPath = current.distance;
46             break;
47         }
48
49         for (int i = 0; i < 4; i++) {
50             int newRow = current.row + dRow[i];
51             int newCol = current.col + dCol[i];
52
53             if (newRow >= 0 && newRow < R && newCol >= 0
54 && newCol < C) {
55                 if (grid[newRow][newCol] == 0 &&
56 !visited[newRow][newCol]) {
57                     visited[newRow][newCol] = true;
58                     q.push({newRow, newCol,
59 current.distance + 1});
60                 }
61             }
62         }
63
64         cout << shortestPath << endl;
65
66         return 0;
67     }

```

B. Output Program



```
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> cd 'c:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output'
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> & .\bfs.exe
8 18
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
```

Gambar 3. Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Mencari jalur tercepat dalam sebuah labirin berbentuk grid dua dimensi yang memiliki halangan membutuhkan pendekatan eksplorasi yang teratur dan merata. Algoritma Breadth-First Search (BFS) digunakan secara khusus dalam kasus ini karena cara kerjanya yang berbasis lapisan, yaitu mengelola elemen secara berurutan mulai dari lapisan teratas hingga lapisan berikutnya, sehingga secara pasti memastikan bahwa titik tujuan yang ditemukan pertama kali merupakan jalur dengan jumlah langkah terkecil.

Secara mekanis, BFS menggunakan struktur data queue untuk menyimpan koordinat sel yang sedang dieksplorasi beserta jarak total yang ditempuh dari titik awal. Proses dimulai dengan memasukkan posisi start ke dalam queue dan menandainya. Selama queue masih berisi elemen, sel yang berada di bagian depan akan diambil. Algoritma selanjutnya memeriksa apakah sel tersebut merupakan titik akhir; jika benar, pencarian langsung dihentikan dan total jarak yang telah ditempuh dikembalikan sebagai solusi akhir. Namun, jika belum ditemukan, pencarian diperluas ke keempat arah yang diperbolehkan, yaitu atas, bawah, kiri, dan kanan. Ekspansi jalur harus melewati beberapa pemeriksaan yang ketat, yaitu koordinat tujuan tidak boleh melebihi batas-batas grid, sel tersebut harus merupakan jalur yang bisa dilalui (bernilai 0), dan sel tersebut belum pernah dikunjungi sebelumnya dalam setiap lintasan yang ada. Untuk mencegah terjadinya infinite loop atau pemeriksaan berulang pada sel yang sama, program menggunakan matriks boolean terpisah yang berfungsi untuk mencatat status kunjungan setiap sel.

Sebagai contoh dalam operasi, jika titik (0,0) sedang dievaluasi dan terdapat jalur terbuka di sisi kanan ke titik (0,1) serta di bawah ke titik (1,0), kedua sel tersebut akan dimasukkan ke dalam antrian dengan jarak yang ditingkatkan menjadi 1. Saat giliran sel (0,1) tiba untuk diekstraksi, tetangga yang sah akan dicatatkan dalam antrian dengan jarak 2. Propagasi ini menyebar seperti gelombang air secara bersamaan ke segala arah hingga mencapai titik tujuan (misalnya sel (7,9) dalam 16 langkah) atau hingga semua opsi jalur yang valid habis tanpa menemukan solusi, yang akan menyebabkan program mengembalikan nilai -1.

Analisis Kompleksitas

1. Kompleksitas Waktu: $O(R \times C)$

Karakteristik dari Breadth-First Search memastikan bahwa setiap sel dalam labirin hanya diproses dan dimasukkan ke dalam queue sekali, berkat adanya matriks yang digunakan untuk merekam kunjungan ke sel tersebut. Evaluasi keempat arah tetangga berjalan dalam waktu yang tetap, yaitu $O(1)$. Berdasarkan batasan maksimum panjang baris $R \leq 100$ dan lebar kolom $C \leq 100$, skenario yang paling berat terjadi ketika seluruh grid terdiri dari jalan kosong, sehingga program harus memeriksa setiap sudutnya, yang menghasilkan jumlah operasi dasar maksimal sekitar $100 \times 100 = 10.000$. Angka ini memiliki beban komputasi yang sangat kecil, sehingga memastikan program selesai jauh sebelum mencapai Time Limit 1.0 s.

2. Kompleksitas Ruang: $O(R \times C)$

Kebutuhan penyimpanan utama digunakan untuk membuat grid dua dimensi yang berfungsi sebagai peta labirin serta matriks boolean dua dimensi untuk mencatat status kunjungan. Selain itu, alokasi memori pada queue berubah secara dinamis, sehingga pada fase pelebaran puncaknya dapat menampung elemen-elemen dengan ukuran yang sebanding dengan lebar dimensi grid. Dengan nilai maksimal R sebesar 100 dan C sebesar 100, total penggunaan memori hanya mencapai beberapa puluh ribu byte data saja. Skala ruang ini sangat kecil dan optimal, sehingga program dapat berjalan dengan lancar tanpa menguras memori dan tidak menimbulkan risiko melebihi batas Memory Limit 64 MB yang ditentukan.

SOAL 4:

Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut:

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat: T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4. Source Code dfs.cpp

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C;
6  int startRow, startCol, endRow, endCol;
7  vector<vector<int>> grid;
8  vector<vector<bool>> visited;
9  int totalPaths = 0;
10
11 int dRow[] = {-1, 1, 0, 0};
12 int dCol[] = {0, 0, -1, 1};
13
14 void dfs(int row, int col) {
15     if (row == endRow && col == endCol) {
16         totalPaths++;
17         return;
18     }
19
20     visited[row][col] = true;
21
22     for (int i = 0; i < 4; i++) {
23         int newRow = row + dRow[i];
24         int newCol = col + dCol[i];
25
26         if (newRow >= 0 && newRow < R && newCol >= 0 &&
27             newCol < C) {
28             if (grid[newRow][newCol] == 0 &&
29                 !visited[newRow][newCol]) {

```

```

28         dfs(newRow, newCol);
29     }
30 }
31 }
32
33     visited[row][col] = false;
34 }
35
36 int main() {
37     cin >> R >> C;
38
39     grid.assign(R, vector<int>(C));
40     visited.assign(R, vector<bool>(C, false));
41
42     for (int i = 0; i < R; i++) {
43         for (int j = 0; j < C; j++) {
44             cin >> grid[i][j];
45         }
46     }
47
48     cin >> startRow >> startCol;
49     cin >> endRow >> endCol;
50
51     dfs(startRow, startCol);
52
53     cout << totalPaths << endl;
54
55     return 0;
56 }

```

B. Output Program



```

PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> cd 'c:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output'
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output> gfs.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree\output>

```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Mendata seluruh kemungkinan jalur dari titik awal ke titik akhir memerlukan program untuk menjelajahi setiap cabang labirin secara komprehensif. Berbeda dari pencarian rute terpendek yang lebih fokus pada penyebaran yang merata, masalah ini mengharuskan penelusuran mendalam hingga ke akhir setiap jalur. Maka dari itu, penggunaan algoritma Depth-First Search (DFS) secara rekursif yang dipadukan dengan teknik Backtracking adalah metode yang sangat penting untuk menghitung semua jalur sederhana yang valid tanpa mengulangi kunjungan.

Algoritma ini berfungsi dengan cara menandai atau mengunci setiap sel yang sedang dijelajahi di dalam matriks boolean yang menunjukkan status kunjungan, dengan tujuan untuk memastikan tidak ada sel yang dilewati lebih dari satu kali dalam lintasan yang sama. Ketika pencarian DFS berhasil mencapai tujuan, penghitung total jalur akan bertambah. Aspek paling penting dalam algoritma ini terdapat pada tahap Backtracking; setiap kali fungsi rekursif menyelesaikan tugasnya, baik karena menemui jalan buntu, mencapai tujuan, atau telah memeriksa semua cabang yang ada, program akan menghapus atau menghilangkan tanda kunjungan pada sel yang sedang dijelajahi. Proses ini mengembalikan status sel grid ke keadaan awal, memberikan peluang bagi kombinasi jalur lainnya untuk menggunakan sel tersebut di penelusuran yang akan datang.

Sebagai contoh, jika penelusuran dimulai dari posisi (0,0) dan bergerak ke cabang kanan (0,1), program akan menandai (0,0) dan (0,1) sebagai ditutup. Setelah seluruh kemungkinan dari (0,1) dievaluasi sepenuhnya secara rekursif, program akan kembali ke (0,0) dan menghapus tanda kunjungan di (0,1). Status grid yang telah di-reset ini memungkinkan algoritma untuk menguji kemungkinan rute alternatif lainnya, seperti bergerak ke bawah menuju (1,0), seolah-olah pergerakan ke arah kanan sebelumnya tidak pernah dilakukan.

Analisis Kompleksitas

1. Kompleksitas Waktu: $O(4^{R \times C})$

Sifat eksplorasi pencarian total dengan DFS dan Backtracking menghasilkan kompleksitas waktu yang melaju secara eksponensial pada batas atas teoretisnya. Pada setiap perpindahan koordinat, fungsi memicu empat panggilan cabang baru ke segala arah. Namun, beban komputasi di lapangan dibatasi secara ekstrem oleh

dimensi grid yang dirancang berukuran maksimum hanya 7×7 sel, serta jaminan bahwa kuantitas rute yang tervalidasi tidak akan melampaui ambang batas 100.000 iterasi penyelesaian. Kalkulasi mendalam ini berpotensi mengeksekusi ratusan ribu instruksi, namun tetap berjalan di atas parameter performa yang memadai untuk menyelesaikan operasi secara utuh sebelum menyentuh batas Time Limit 2.0 s.

2. Kompleksitas Ruang: $O(R \times C)$

Pemakaian sumber daya memori dialokasikan secara spesifik untuk membentuk grid data utama dan matriks rekam jejak, yang masing-masing hanya berkapasitas maksimal 7×7 elemen. Karena arsitektur pencarian dieksekusi secara rekursif, kedalaman tumpukan (Call Stack) di dalam memori program bertumbuh proporsional terhadap panjang jejak rute sederhana yang ditarik secara berkesinambungan, yang secara matematis tidak akan melebihi total sel keseluruhan yaitu $R \times C$ iterasi tertunda. Manajemen ruang memori berskala kecil ini mempresentasikan stabilitas tinggi dan sama sekali tidak menimbulkan risiko meluapnya kapasitas Memory Limit yang ditetapkan pada angka 64 MB.

SOAL 5: Tree Traversal

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut:

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut:

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut:

[Preorder] : daftar_angka
[Inorder] : daftar_angka
[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 20 40 70 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5. Source Code treeTraversal.cpp

1	#include <iostream>
2	#include <string>
3	#include "RedBlackTree.h"
4	using namespace std;
5	

```

6 void printPreorder(const RedBlackTree::Node* node, const
  RedBlackTree::Node* nilNode) {
7     if (node == nilNode) return;
8
9     cout << " " << node->key;
10    printPreorder(node->left, nilNode);
11    printPreorder(node->right, nilNode);
12 }
13
14 void printInorder(const RedBlackTree::Node* node, const
  RedBlackTree::Node* nilNode) {
15     if (node == nilNode) return;
16
17    printInorder(node->left, nilNode);
18    cout << " " << node->key;
19    printInorder(node->right, nilNode);
20 }
21
22 void printPostorder(const RedBlackTree::Node* node,
  const RedBlackTree::Node* nilNode) {
23     if (node == nilNode) return;
24
25    printPostorder(node->left, nilNode);
26    printPostorder(node->right, nilNode);
27    cout << " " << node->key;
28 }
29
30 int main() {
31     int n;
32     cin >> n;
33
34     RedBlackTree tree;
35
36     for (int i = 0; i < n; i++) {
37         int val;
38         cin >> val;
39
40         if (!tree.contains(val)) {
41             tree.insert(val);
42         }
43     }
44 }

```

```

45     int q;
46     cin >> q;
47
48     for (int i = 0; i < q; i++) {
49         string query;
50         cin >> query;
51
52         if (tree.empty()) {
53             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan." << endl;
54             continue;
55         }
56
57         const RedBlackTree::Node* rootNode =
tree.root();
58         const RedBlackTree::Node* nilNode = tree.nil();
59
60         if (query == "PREORDER" || query == "ALL") {
61             cout << "[Preorder] :";
62             printPreorder(rootNode, nilNode);
63             cout << endl;
64         }
65         if (query == "INORDER" || query == "ALL") {
66             cout << "[Inorder] :";
67             printInorder(rootNode, nilNode);
68             cout << endl;
69         }
70         if (query == "POSTORDER" || query == "ALL") {
71             cout << "[Postorder]:";
72             printPostorder(rootNode, nilNode);
73             cout << endl;
74         }
75     }
76
77     return 0;
78 }

```

B. Output Program

```
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree> g++ treeTraversal.cpp RedBlackTree.cpp -o traversal ; .\traversal
7
50 30 20 40 70 60 80
4
PREORDER
[Preorder] : 30 20 50 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder]: 20 40 60 80 70 50 30
ALL
[Preorder] : 30 20 50 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 60 80 70 50 30
PS C:\Users\Lenovo\Documents\GitHub\module-6-graph-and-tree>
```

Gambar 5. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Struktur data Red-Black Tree (RBT) difungsikan untuk menyimpan sekumpulan nilai secara dinamis dengan sifat self-balancing, sehingga menjamin tinggi pohon (tree height) tetap proporsional dan tidak condong ke satu sisi. Pengurutan nilai yang konsisten dan seimbang ini memerlukan mekanisme traversal yang sistematis untuk mengakses kembali data di dalamnya. Oleh karena itu, program dirancang untuk membaca serangkaian N input bilangan bulat, memfilter duplikasi secara langsung sebelum penyisipan, dan menyusunnya ke dalam RBT, untuk kemudian merespons berbagai query pencetakan sesuai format yang diminta.

Proses pembentukan RBT diawali dengan perulangan yang memverifikasi setiap angka menggunakan fungsi pencarian bawaan (`contains`). Jika nilai belum eksis di dalam tree, operasi penyisipan (`insert`) akan dieksekusi. Sifat RBT secara otomatis akan mengatur rotasi dan pewarnaan node di balik layar untuk menjaga keseimbangan hierarki. Setelah struktur terbentuk sempurna, algoritma beralih membaca daftar query. Apabila tree ternyata kosong (misalnya karena tidak ada data valid yang tersisa), program akan langsung mencetak pesan penolakan khusus. Sebaliknya, jika data tersedia, program akan melakukan penelusuran rekursif. Metode PREORDER mencetak nilai saat node pertama kali dikunjungi sebelum cabang turun, INORDER mencetak nilai di antara kunjungan cabang kiri dan kanan sehingga menghasilkan deret terurut, sedangkan POSTORDER menyelesaikan seluruh cabang bawahan terlebih dahulu sebelum mencetak nilai utama node tersebut.

Sebagai ilustrasi, apabila angka 50, 30, dan 70 disisipkan, nilai 50 akan didapat menjadi root utama. Angka 30 yang lebih kecil masuk ke percabangan subtree kiri, sedangkan 70 yang lebih besar masuk ke percabangan subtree kanan. Saat sebuah query INORDER diminta, fungsi rekursif akan menuruni cabang kiri terlebih dahulu untuk mencetak 30, kembali ke atas untuk mencetak 50, lalu pindah ke kanan untuk mencetak 70, menghasilkan luaran yang berurutan secara linier.

Analisis Kompleksitas

1. Kompleksitas Waktu: $O(N \log N + Q \times N)$

Operasi pencarian ada tidaknya data (`contains`) dan penyisipan data baru (`insert`) pada Red-Black Tree masing-masing beroperasi dalam kompleksitas logaritmik sebesar $O(\log N)$. Untuk memasukkan seluruh N elemen, waktu komputasi yang ditarik adalah $O(N \log N)$. Selanjutnya, eksekusi setiap query traversal menuntut fungsi rekursif untuk mengunjungi setiap node tepat satu kali, menghasilkan kompleksitas waktu linear $O(N)$ per pemanggilan query. Mengingat batas maksimal data hanyalah $N \leq 1000$ dan jumlah kueri dibatasi $Q \leq 20$, seluruh rangkaian algoritma, baik fase pembangunan pohon maupun rentetan fase pencetakan, hanya memakan sangat sedikit kalkulasi yang dijamin akan terselesaikan sekejap, melampaui batasan Time Limit 1.0 s.

2. Kompleksitas Ruang: $O(N)$

Setiap entitas nilai baru yang dialokasikan ke dalam RBT akan membentuk sebuah node dinamis yang memuat atribut kunci, warna, serta kumpulan pointer referensi. Alokasi ini bertumbuh sejalan dengan jumlah input dan terakumulasi hingga memuat N elemen. Pemrosesan pencetakan berbasis rekursi juga akan mengonsumsi memori tumpukan eksekusi (Call Stack) sementara, namun kedalamannya tertahan secara logaritmik $O(\log N)$ berkat kemampuan pohon untuk menyeimbangkan diri secara mandiri. Dengan parameter yang amat kecil yakni $N \leq 1000$, memori yang dialokasikan untuk menyimpan node dan tumpukan eksekusi sangat ringan dan statis, sehingga sama sekali tidak berisiko membobol langit-langit Memory Limit 64 MB yang disyaratkan.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-alyanriz>